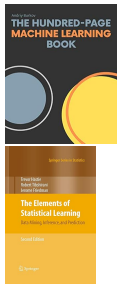


Lesson 04.2: Machine Learning Models

Instructor: Dr. GP Saggese, gsaggese@umd.edu

References:

- Burkov: “*Machine Learning Engineering*” (2020)
- Hastie et al.: “*The Elements of Statistical Learning*” (2nd ed, 2009)



- *Logistic Regression*
- LDA, QDA
- Kernel Methods
- Support Vector Machines (Optional)

Logistic Regression Is a Probabilistic Classifier

- **Logistic regression** learns:
 - The probability of each class $\Pr(y|\underline{x})$ given input $\underline{x}$
 - Instead of predicting class y directly
- **Parametric approach**: assume $\Pr(y|\underline{x}; \underline{w})$ has a known functional form

$$\Pr(y = 1|\underline{x}; \underline{w}) = \text{logit}(\underline{w}^T \underline{x})$$

- **Optimize parameters** $\underline{w}$ using maximum likelihood

$$\underline{w}^* = \operatorname{argmax}_{\underline{w}} \Pr(\mathcal{D}; \underline{w})$$

- **Predict** by outputting class with highest probability

$$h_{\underline{w}}(\underline{x}) = \begin{cases} +1 & \Pr(y = 1|\underline{x}; \underline{w}) \geq 0.5 \\ -1 & \Pr(y = 1|\underline{x}; \underline{w}) < 0.5 \end{cases}$$

Logistic Regression: Example

- Assume y is:
 - The event y_i “*patient with characteristics $\underline{x}$ had heart attack*”
 - Function of parameters $\underline{x}$ (e.g., age, gender, diet)
- In data set $\mathcal{D}$:
 - No samples of $\Pr(y|\underline{x})$
 - Have realizations:
 - * “*Patient with $\underline{x}_1$ had a heart attack*”
 - * “*Patient with $\underline{x}_2$ didn’t*”
 - * ...
- Learn $\Pr(y|\underline{x})$
 - Find best parameters $\underline{w}$ for logistic regression model to explain data $\mathcal{D}$

Logistic Function

- Aka “sigmoid”
- **Logistic function** $\text{logit}(s)$ is defined as

$$\text{logit}(s) \triangleq \frac{e^s}{1 + e^s} = \frac{1}{1 + e^{-s}}$$

- Varies in $[0, 1]$
- Crosses the origin at 0.5
- Asymptotes at 0 and 1
- It is a soft version of $\text{sign}()$

Logistic Regression vs Linear Classifier

- **Functional form is similar**
 - **Logistic regression**: $h(\underline{\mathbf{w}}) = \text{logit}(\underline{\mathbf{w}}^T \underline{\mathbf{x}})$
 - **Linear classifier** (perceptron): $h(\underline{\mathbf{w}}) = \text{sign}(\underline{\mathbf{w}}^T \underline{\mathbf{x}})$
- **Difference** in probabilistic interpretation and fitting method
 - Logistic regression** lacks samples of probability function to interpolate
 - * Uses realizations of random variable
 - * Seeks model parameters maximizing data likelihood
 - **Linear classification** assumes class value is linear function of inputs

Error for Probabilistic Binary Classifiers

- For probabilistic binary classification, use **log-probability error** as point-wise error

$$e(h(\underline{\mathbf{x}}), y) \triangleq -\log(\Pr(y = h(\underline{\mathbf{x}})|\underline{\mathbf{x}}; \underline{\mathbf{w}}))$$

- Negate for positive errors: $\log([0, 1]) \in [-\infty, 0)$
- Log probability generalizes 0-1 error
 - Case $y = 1$
 - * Output $h(\underline{\mathbf{x}})$ close to 1 $\implies$ probability 1 $\implies \log(1) = 0$
 $\implies e(\cdot) = 0$
 - * Output close to 0 $\implies e() = -\log(0) \rightarrow +\infty$
 - Similar behavior for $y = 0$

One-Liner Error for Probabilistic Binary Classifiers

- **Point-wise error** for example $(\underline{x}, y)$ for probabilistic binary classifiers is defined as:

$$\begin{aligned} e(h(\underline{x}), y) &\triangleq -\log(\Pr(h(\underline{x}) = y|\underline{x})) \\ &= \begin{cases} -\log(\Pr(y = 1|\underline{x})) & y = 1 \\ -\log(\Pr(y = 0|\underline{x})) & y = 0 \end{cases} \\ &= \begin{cases} -\log(\Pr(y = 1|\underline{x})) & y = 1 \\ -\log(1 - \Pr(y = 1|\underline{x})) & y = 0 \end{cases} \end{aligned}$$

- Any function of a binary variable:

$$y = \begin{cases} a & x = 1 \\ b & x = 0 \end{cases}$$

can be written as one-liner: $y = x \cdot a + (1 - x) \cdot b$

- Point-wise error can be written independently of $\Pr(y = 1|\underline{x})$:

$$e(h(\underline{x}), y) = -y \log(\Pr(y = 1|\underline{x})) - (1 - y) \log(1 - \Pr(y = 1|\underline{x}))$$

One-Liner Error for Logistic Regression

- The point-wise error for a binary classifier is:

$$e(h(\underline{\mathbf{x}}), y) = -y \log(\Pr(y = 1|\underline{\mathbf{x}})) - (1 - y) \log(1 - \Pr(y = 1|\underline{\mathbf{x}}))$$

- Simplify further **with logit function**:

$$e(h(\underline{\mathbf{x}}), y) \triangleq -\log \Pr(h(\underline{\mathbf{x}}) = y)$$

... a bit of math manipulation ...

$$= -\log \text{logit}(y \underline{\mathbf{w}}^T \underline{\mathbf{x}})$$

$$\text{since } \text{logit}(s) = \frac{1}{1 + e^{-s}}$$

$$= \log(1 + \exp(-y \underline{\mathbf{w}}^T \underline{\mathbf{x}}))$$

- Point-wise error for logistic regression equals **cross-entropy error**

Cross-Entropy Error

- **Point-wise error for logistic regression** has expression:

$$e(h(\underline{\mathbf{x}}), y) = \log(1 + \exp(-y \cdot \underline{\mathbf{w}}^T \underline{\mathbf{x}}))$$

- It is called **cross-entropy error**
- Note: no – before $\log(\cdot)$ but before $y \cdot \underline{\mathbf{w}}^T \underline{\mathbf{x}}$
- Cross-entropy error **generalizes 0-1 error**
 - If $\underline{\mathbf{w}}^T \underline{\mathbf{x}}$ agrees with y in sign and $|\underline{\mathbf{w}}^T \underline{\mathbf{x}}|$ is large $\implies$ error goes to 0
 - If they disagree in sign $\implies$ error goes towards ∞
- Define **in-sample error on training set** as average of point-wise errors:

$$E_{in} \triangleq \frac{1}{N} \sum_n e(h(\underline{\mathbf{x}}_n), y_n)$$

Fitting Logistic Regression

- A plausible error measure of a hypothesis is based on **likelihood of data**

$$\Pr(\mathcal{D}|h = f)$$

- “How likely is the data $\mathcal{D}$ under the assumption that $h = f$?”
- “How likely is that the data $\mathcal{D}$ was generated by h ?”

- **Maximize likelihood** $\mathcal{D}$ generated from logistic regression

$$\Pr(y = 1|\underline{\mathbf{x}}; \underline{\mathbf{w}})$$

- It can be proved that this is equivalent to **minimizing in-sample error** on training set **using cross-entropy error**

Fitting Logistic Regression (Optional)

- Find $\underline{\mathbf{w}}$ that maximizes likelihood for data set $\mathcal{D} = \{(\underline{\mathbf{x}}_1, y_1), \dots, (\underline{\mathbf{x}}_N, y_N)\}$ generated by model $h(\underline{\mathbf{x}})$:

$$\Pr(D|\underline{\mathbf{w}}) = \Pr(y_1 = h(\underline{\mathbf{x}}_1) \wedge \dots \wedge y_N = h(\underline{\mathbf{x}}_N)) = \Pr(y_1 = y'_1 \wedge \dots \wedge y_N = y'_N)$$

- Model form:

$$y' = h(\underline{\mathbf{x}}) = \begin{cases} +1 & \text{if } \text{logit}(\underline{\mathbf{w}}^T \underline{\mathbf{x}}) > 0.5 \\ -1 & \text{otherwise} \end{cases}$$

- Assuming independence among training examples:

$$\Pr(D|\underline{\mathbf{w}}) = \prod_{i=1}^N \Pr(y_i = y'_i | \underline{\mathbf{x}}_i)$$

- Fold y_n in expression:

- When $y_n = 1$, $\Pr(y_n = y'_n) = \text{logit}(\underline{\mathbf{w}}^T \underline{\mathbf{x}}_n)$
- When $y_n = -1$, $\Pr(y_n = y'_n) = \text{logit}(-\underline{\mathbf{w}}^T \underline{\mathbf{x}}_n)$

- Thus, $\Pr(y_n = y'_n) = \text{logit}(y_n \underline{\mathbf{w}}^T \underline{\mathbf{x}}_n)$

Fitting Logistic Regression (Optional)

- Given:

$$\Pr(D|\underline{\mathbf{w}}) = \prod_{i=1}^N \text{logit}(y_n \underline{\mathbf{w}}^T \underline{\mathbf{x}}_n)$$

- Re-write optimization as minimizing sum of point-wise errors
 $E_{in} = \sum e(h(\underline{\mathbf{x}}_n), y_n)$
 - Maximize $\log(\dots)$ with respect to $\underline{\mathbf{w}}$ since $\log$ argument > 0 and $\log()$ is monotone
- Equivalently, minimize:

$$\begin{aligned} -\frac{1}{N} \log(\dots) &= -\frac{1}{N} \log(\prod(\dots)) = -\frac{1}{N} \sum (\log(\dots)) \\ &= \frac{1}{N} \sum \log\left(\frac{1}{\text{logit}(y_n \underline{\mathbf{w}}^T \underline{\mathbf{x}}_n)}\right) \\ &= \frac{1}{N} \sum \log(1 + \exp(-y_n \underline{\mathbf{w}}^T \underline{\mathbf{x}}_n)) = \frac{1}{N} \sum e(h(\underline{\mathbf{x}}_n), y_n) = E_{in}(\underline{\mathbf{w}}) \end{aligned}$$

Gradient Descent for Logistic Regression

- Gradient descent requires two inputs:
 - Gradient of the cost function $\frac{\partial E}{\partial w_j}$ for all j
 - Cost function $E_{in}(\underline{\mathbf{w}})$

- The cost function is:

$$E_{in}(\underline{\mathbf{w}}) = \frac{1}{N} \sum_i e(h(\underline{\mathbf{x}}_i; \underline{\mathbf{w}}), y_i)$$

- The cost function for logistic regression:

$$E_{in}(\underline{\mathbf{w}}) = \frac{1}{N} \sum_i \log(1 + \exp(-y_i \underline{\mathbf{w}}^T \underline{\mathbf{x}}_i))$$

- Thus gradient descent converges to global minimum
 - It can be shown that $E_{in}(\underline{\mathbf{w}})$ is convex in $\underline{\mathbf{w}}$
 - In fact sum of exponentials and flipped exponentials is convex and log is monotone

One-Vs-All Multi-Class Classification

- Aka “one-vs-rest” classifier
- **Problem:** you have n classes $c_1, \dots, c_n$ to distinguish given $\underline{x}$
- **Learn**
 - Create n binary classification problems where we classify c_i vs c_{-i} (everything but i)
 - Learn n classifiers with optimal $\underline{w}_i$, each estimating $\Pr(y = i | \underline{x}; \underline{w}_i)$
- **Predict**
 - Evaluate the n classifiers
 - Pick the class $y = i$ with the maximum $\Pr(y = i | \underline{x})$

Cost Function for Multi-Class Classification (Opt)

- The cost function for logistic regression is:

$$E_{in}(\underline{\mathbf{w}}) = -\frac{1}{N} \sum_{i=1}^N (y_i \log \Pr(y = 1|\underline{\mathbf{x}}_i) + (1 - y_i) \log(1 - \Pr(y = 1|\underline{\mathbf{x}}_i)))$$

- Encode expected outputs $\underline{\mathbf{y}}_i$ one-hot
 - j -th element $\underline{\mathbf{y}}_i|_j$ is 1 if correct class is j -th
 - E.g., for $k = 4$ 1000
- Using $\underline{\mathbf{h}}(\underline{\mathbf{x}})$ as model outputs and $\Pr(y = 1|\underline{\mathbf{x}}) = p(\underline{\mathbf{x}})$:

$$E_{in}(\underline{\mathbf{w}}) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K \left(\underline{\mathbf{y}}_i \log(\underline{\mathbf{p}}(\underline{\mathbf{x}}_i)) + (1 - \underline{\mathbf{y}}_i) \log(1 - \underline{\mathbf{p}}(\underline{\mathbf{x}}_i)) \right) |_k$$

- Innermost summation considers error on each class/digit
 - E.g., for $k = 4$ 1000 vs 0100
 - Equal digits don't contribute to error
 - Different digits give positive contribution

- Logistic Regression
- **LDA, QDA**
- Kernel Methods
- Support Vector Machines (Optional)

Basic Idea of Parametric Models

- Assume a model generates data
 - Known functional form
 - Parametrized with unknown parameters to estimate
- **Pros**
 - Utilize data structure
 - Easy to fit: few parameters
 - Accurate predictions if assumptions correct
- **Cons**
 - Strong data assumptions
 - Low accuracy if assumptions incorrect

Linear and Quadratic Discriminant Analysis

- Aka LDA and QDA
- **Parametric models**
 - Assume each class generating process is multivariate Gaussian
 - Classifiers with linear and quadratic decision surface
- **Pros**
 - Closed-form solutions easy to compute (sample mean and covariance)
 - Inherently multiclass
 - No hyperparams to tune
- **Cons**
 - Strong assumptions about the data

LDA / QDA: Model Form

- Both LDA and QDA assume class generating process $f_k(\underline{\mathbf{x}}; \underline{\boldsymbol{\mu}}_k, \underline{\boldsymbol{\Sigma}}_k)$ is **multivariate Gaussian**
- Linear discriminant analysis (LDA)** model:

$$f_k(\underline{\mathbf{x}}; \underline{\boldsymbol{\mu}}_k, \underline{\boldsymbol{\Sigma}}_k) \sim \mathcal{N}(\underline{\boldsymbol{\mu}}_k, \underline{\boldsymbol{\Sigma}})$$

- Means $\underline{\boldsymbol{\mu}}_k$ differ for all k classes
 - Covariance matrix $\underline{\boldsymbol{\Sigma}}$ same for all k classes
 - Classes separated by linear decision boundaries
- Quadratic discriminant analysis (QDA)** model:
 - Classes k have different covariance matrix $\underline{\boldsymbol{\Sigma}}_k$
 - Classes separated by quadratic boundaries

Bayes Theorem for LDA / QDA

- Consider a classification setup with multi-class output $Y \in \{1, \dots, K\}$
- Build a **parametric model for the conditional distribution**:

$$\Pr(Y = k | X = \underline{x})$$

- Use **Bayes theorem**:

$$\Pr(Y = k | X = \underline{x}) = \frac{\Pr(X = \underline{x} | Y = k) \cdot \Pr(Y = k)}{\Pr(X = \underline{x})}$$

where:

- $\Pr(X = \underline{x} | Y = k)$: given a class, estimate probability of $\underline{x}$
 - $\Pr(Y = k) = \pi_k$: probability of each class (prior)
 - $\Pr(X = \underline{x})$: probability of each input
- **Estimate probabilities** from data

LDA / QDA: Boundary Decision (Optional)

- Consider the ratio between the probabilities of $Y = k$ vs $Y = j$:

$$r = \frac{\Pr(Y = k|X = \underline{\mathbf{x}})}{\Pr(Y = j|X = \underline{\mathbf{x}})}$$

- Using the model assumption and Bayes theorem:

$$\begin{aligned}\Pr(Y = k|X = \underline{\mathbf{x}}) &\propto \Pr(X = \underline{\mathbf{x}}|Y = k) \cdot \Pr(Y = k) \\ &= f_k(\underline{\mathbf{x}}; \underline{\boldsymbol{\mu}}_k) \cdot \pi_k\end{aligned}$$

where:

$$f_k(\underline{\mathbf{x}}) = c \cdot \exp\left(-\frac{1}{2}(\underline{\mathbf{x}} - \underline{\boldsymbol{\mu}}_k)^T \underline{\boldsymbol{\Sigma}}^{-1}(\underline{\mathbf{x}} - \underline{\boldsymbol{\mu}}_k)\right)$$

- Apply $\log(\cdot)$ as a monotone transformation

$$r = \log \frac{f_k(\underline{\mathbf{x}})}{f_j(\underline{\mathbf{x}})} + \log \frac{\pi_k}{\pi_j}$$

LDA / QDA: Boundary Decision (Optional)

- Apply $\log(\cdot)$ as a monotone transformation

$$r = \log \frac{f_k(\underline{\mathbf{x}})}{f_j(\underline{\mathbf{x}})} + \log \frac{\pi_k}{\pi_j}$$

- **Second term** independent of $\underline{\mathbf{x}}$ so you can ignore it
- **First term** proportional to:

$$(\underline{\mathbf{x}} - \underline{\boldsymbol{\mu}}_k)^T \underline{\boldsymbol{\Sigma}}^{-1} (\underline{\mathbf{x}} - \underline{\boldsymbol{\mu}}_k) - (\underline{\mathbf{x}} - \underline{\boldsymbol{\mu}}_j)^T \underline{\boldsymbol{\Sigma}}^{-1} (\underline{\mathbf{x}} - \underline{\boldsymbol{\mu}}_j)$$

- Expand, simplify $\underline{\mathbf{x}}^T \underline{\boldsymbol{\Sigma}}^{-1} \underline{\mathbf{x}}$, and note:

$$2\underline{\mathbf{x}}^T \underline{\boldsymbol{\Sigma}}^{-1} (\underline{\boldsymbol{\mu}}_k - \underline{\boldsymbol{\mu}}_j)$$

Resulting in a linear relationship in $\underline{\mathbf{x}}$

$$-\frac{1}{2}(\underline{\boldsymbol{\mu}}_k + \underline{\boldsymbol{\mu}}_j)^T \underline{\boldsymbol{\Sigma}}^{-1} (\underline{\boldsymbol{\mu}}_k - \underline{\boldsymbol{\mu}}_j) + \underline{\mathbf{x}}^T \underline{\boldsymbol{\Sigma}}^{-1} (\underline{\boldsymbol{\mu}}_k - \underline{\boldsymbol{\mu}}_j)$$

LDA / QDA: Learn Model (Optional)

- In practice:
 - Ignore $\Pr(X = \underline{x})$; it's common for all classes
 - Know or estimate prior $\Pr(Y = k) = \pi_k$ from data
 - Estimate conditional probability $\Pr(X = \underline{x} | Y = k)$
- Model assumes Gaussian distribution for conditional probability:

$$\Pr(X = \underline{x} | Y = k) = f_k(\underline{x}; \underline{\mu}_k, \underline{\Sigma}_k)$$

where:

$$f_k(\underline{x}) = \frac{1}{(2\pi)^n |\underline{\Sigma}_k|^{1/2}} \exp\left(-\frac{1}{2}(\underline{x} - \underline{\mu}_k)^T \underline{\Sigma}_k^{-1}(\underline{x} - \underline{\mu}_k)\right)$$

- Estimate parameters $\underline{\mu}_k, \underline{\Sigma}_k$ using sample mean and covariance

Evaluating LDA / QDA

- For new $\underline{x} = \underline{x}'$, compute for each class $Y = k$

$$\Pr(Y = k|X = \underline{x}) \propto f_k(\underline{x}; \underline{\mu}_k, \underline{\Sigma}_k) \cdot \pi_k$$

- Choose k that maximizes posterior probability
- If $f_i(\underline{x})$ is from a multivariate Gaussian distribution with diagonal $\underline{\Sigma}$ (feature independence), you can simplify expressions further

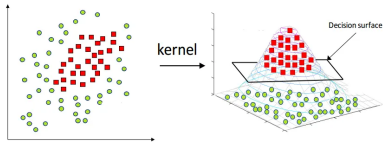
- Logistic Regression
- LDA, QDA
- ***Kernel Methods***
- Support Vector Machines (Optional)

Kernel: Definition

- Consider a transformation $\Phi : \mathcal{X} \rightarrow \mathcal{Z}$
 - E.g., transform features in space $\mathcal{X}$ non-linearly into higher-dimensional space $\mathcal{Z}$
- **Kernel of transformation** Φ yields inner product of two points $\underline{x}, \underline{x}' \in \mathcal{X}$ in transformed space $\mathcal{Z}$

$$K_{\Phi}(\underline{x}, \underline{x}') \triangleq \langle \Phi(\underline{x}), \Phi(\underline{x}') \rangle = \Phi(\underline{x})^T \Phi(\underline{x}') = \underline{z}^T \underline{z}'$$

- Why doing this?



Kernel: Expression From the Transform

- If you have an expression for Φ , compute a closed formula for the kernel
- E.g., if transformation is $\Phi : \mathbb{R}^2 \rightarrow \mathbb{R}^6$, it introduces interaction terms:

$$\underline{z} = \Phi(\underline{x}) = \Phi(x_1, x_2) = (1, x_1, x_2, x_1^2, x_2^2, x_1x_2)$$

- Kernel of Φ is:

$$\begin{aligned} K_{\Phi}(\underline{x}, \underline{x}') &= (1, x_1, x_2, x_1^2, x_2^2, x_1x_2)^T (1, x'_1, x'_2, x'^2_1, x'^2_2, x'_1x'_2) \\ &= 1 + x_1x'_1 + x_2x'_2 + x_1^2x'^2_1 + x_2^2x'^2_2 + x_1x_2x'_1x'_2 \end{aligned}$$

Gaussian Kernel

- Aka “exponential kernel” or “Radial Basis Function” (RBF) kernel
- A **Gaussian kernel** has the form:

$$K(\underline{\mathbf{x}}, \underline{\mathbf{x}}') = \exp(-\gamma \|\underline{\mathbf{x}} - \underline{\mathbf{x}}'\|^2) = \exp\left(-\frac{\|\underline{\mathbf{x}} - \underline{\mathbf{x}}'\|^2}{\sigma^2}\right)$$

- It can be shown to be an inner product in an infinite dimension $\mathcal{Z}$

Kernel as Way to Measure Similarity

- **Intuition:** The Gaussian kernel

$$K(\underline{\mathbf{x}}, \underline{\mathbf{x}}') = \exp(-\gamma \|\underline{\mathbf{x}} - \underline{\mathbf{x}}'\|^2)$$

measures “similarity” of point $\underline{\mathbf{x}}$ to point $\underline{\mathbf{x}}_i$:

- $K(\underline{\mathbf{x}}, \underline{\mathbf{x}}')$ is 1 when points are the same
- Value is 0 when points are distant
- Effect strength depends on γ
- Using kernels to compute features:
 - Kernels often rely on distance between vectors
 - * E.g., euclidean norm $\|\underline{\mathbf{x}} - \underline{\mathbf{x}}'\|^2$
 - Need to scale features for similar effects among coordinates

Linear Kernel

- Consider the transformation Φ as the identity function $\Phi(\underline{\mathbf{x}}) = \underline{\mathbf{x}}$
- The kernel function is:

$$K_{\Phi}(\underline{\mathbf{x}}, \underline{\mathbf{x}}') = \underline{\mathbf{x}}^T \underline{\mathbf{x}}'$$

- A **linear kernel** means using no kernel
- It is just a “pass-through”

Polynomial Kernel

- Given a point $\underline{\mathbf{x}} \in \mathbb{R}^n$, consider the function with two parameters k and d

$$K_{\Phi}(\underline{\mathbf{x}}, \underline{\mathbf{x}}') = (k + \underline{\mathbf{x}}^T \underline{\mathbf{x}}')^d$$

- It is called **polynomial** since if you expand the dot product you get a polynomial
- It can be proved that this is always a kernel

Kernel: Identifying a Function as a Kernel

- **Problem:**

- You have a certain function $K(\underline{\mathbf{x}}, \underline{\mathbf{x}}')$ and you want to show that $K(\cdot)$ is an inner product in the form for some function $\Phi(\cdot)$

$$K(\underline{\mathbf{x}}, \underline{\mathbf{x}}') = \Phi(\underline{\mathbf{x}})^T \Phi(\underline{\mathbf{x}}') \quad \forall \underline{\mathbf{x}}, \underline{\mathbf{x}}'$$

for a certain Φ and $\mathcal{Z}$

- In theory, a given function $K(\underline{\mathbf{x}}, \underline{\mathbf{x}}')$ is a valid kernel iff:
 - It is a symmetric, and
 - Satisfies the Mercer's condition: the matrix $K(\underline{\mathbf{x}}_i, \underline{\mathbf{x}}_j)$ is definite semi-positive

Kernel: Example of Identifying a Kernel

- Let's show that:

$$K(\underline{\mathbf{x}}, \underline{\mathbf{x}}') = (k + \underline{\mathbf{x}}^T \underline{\mathbf{x}}')^d$$

is a **kernel** for any n, k, d

- According to the definition you need to show that there is always a transform Φ :

$$\Phi : \mathcal{X} = \mathbb{R}^n \rightarrow \mathcal{Z} = \mathbb{R}^q$$

with $q \gg d$, such that $K_\Phi = (k + \underline{\mathbf{x}}^T \underline{\mathbf{x}}')^d$

- Example**

- $\mathcal{X} = \mathbb{R}^2$, $K(\underline{\mathbf{x}}, \underline{\mathbf{x}}') = (1 + \underline{\mathbf{x}}^T \underline{\mathbf{x}}')^2 = (1 + x_1 x'_1 + x_2 x'_2)^2$
- Compute the full expression in terms of the coordinates:

$$K(\underline{\mathbf{x}}, \underline{\mathbf{x}}') = (1 + x_1^2 x'^2_1 + x_2^2 x'^2_2 + 2x_1 x'_1 + 2x_2 x'_2 + 2x_1 x'_1 x_2 x'_2)$$

- Choose:
 - * $\mathcal{Z} = \mathbb{R}^6$
 - * $\Phi(x_1, x_2) = (1, x_1^2, x_2^2, \sqrt{2}x_1, \sqrt{2}x_2, \sqrt{2}x_1 x_2)$
- This is a particular case of the polynomial kernel

A Kernel Is a Computational Shortcut

- In literature, the **kernel trick** is a **computational shortcut** for the dot product of transformed vectors
- Compare 2 ways to compute the inner product of transformed vectors for a polynomial kernel
 1. **Using definition**: compute images of vectors, then inner product in transformed space:

$$(1, x_1, x_2, \sqrt{2}x_1x_2, x_1^2, x_2^2, \dots)^T \cdot (1, x'_1, \dots)$$

- Requires combinatorial powers and a large dot product

2. **Kernel trick**: use kernel function for dot product in transformed space

$$(k + \underline{\mathbf{x}}^T \underline{\mathbf{x}}')^d$$

- Requires inner product of small vectors, then power of a number

- Kernel trick is more computationally efficient for inner product computation

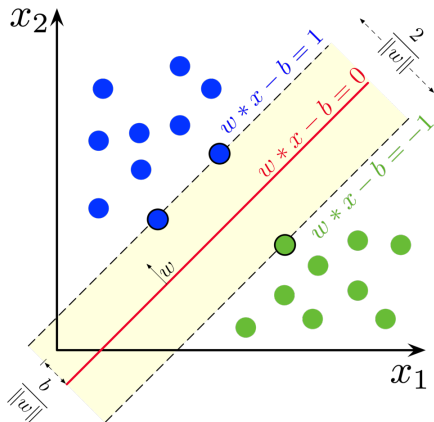
- Logistic Regression
- LDA, QDA
- Kernel Methods
- ***Support Vector Machines (Optional)***

Support Vector Machines (SVM)

- Arguably one of the most successful classification algorithm, together with neural networks and random forests
- **Idea**: find a separating hyperplane that maximizes the distance from the class points (aka “margin”)
- **All the rage in 2005-2015**
 - Robust classifier handling outliers automatically
 - Strong theoretical justification of out-of-bound error
 - Strong link with VC dimension
 - Cool geometric interpretation
 - Solve a very complex optimization problem with some neat tricks
 - Works for both regression and classification
- SVM for classification:
 - Does not output probabilities (like logistic regression), but predicts directly the class
 - Has a notion of confidence, as distance from the margin

SVM Is a Large Margin Classifier

- Why **large margin** classifier is good?
- Given a linearly separable data set, the optimal separating line maximizes the margin:
 - More robust to noise
 - Large margin reduces VC dimension of hypothesis set



SVM: Notation and Conventions

- Assume that:
 1. Outputs are encoded as $y_i \in \{-1, 1\}$
 2. Pull out w_0 from $\underline{\mathbf{w}}$
 - The bias $w_0 = b$ plays a different role
 - $\underline{\mathbf{w}} = (w_1, \dots, w_d)$ and there is no $x_0 = 1$
 - $\underline{\mathbf{w}}^T \underline{\mathbf{x}} + b = 0$ is the equation of the separating hyperplane
 3. $\underline{\mathbf{x}}_n$ is the closest point to the hyperplane
 - It can be multiple points from different classes
- Normalize $\underline{\mathbf{w}}$ and b to get a canonical representation of the hyperplane imposing $|\underline{\mathbf{w}}^T \underline{\mathbf{x}}_n + b| = 1$

SVM: Original Form of Problem

- The SVM problem is:

$$\begin{aligned} & \text{find } \underline{\mathbf{w}}, b \\ & \text{maximize } \frac{1}{\|\underline{\mathbf{w}}\|} && (\text{max margin}) \\ & \text{subject to } \min_{i=1,\dots,n} |\underline{\mathbf{w}}^T \underline{\mathbf{x}}_i + b| = 1 \quad (\text{hyperplane}) \end{aligned}$$

- This problem is not friendly to optimization since it has norm, min, and absolute value

Primal Form of SVM Problem

- You can rewrite it as:

$$\begin{array}{ll}\text{find} & \underline{\mathbf{w}}, b \\ \text{minimize} & \frac{1}{2} \underline{\mathbf{w}}^T \underline{\mathbf{w}} \\ \text{subject to} & y_i(\underline{\mathbf{w}}^T \underline{\mathbf{x}}_i + b) \geq 1 \quad \forall i = 1, \dots, n\end{array}$$

- Note that under $\underline{\mathbf{w}}$ minimal and linear separable classes, it is guaranteed that for at least one $\underline{\mathbf{x}}_i$ in the second equation will be equal to 1 (as in the original problem)
 - In fact otherwise we could scale down $\underline{\mathbf{w}}$ and b (which does not change the plane) to use the slack, against the hypothesis of minimality of $\underline{\mathbf{w}}$

Dual (Lagrangian) Form of SVM Problem

minimize with respect to $\underline{\alpha}$

$$\mathcal{L}(\underline{\alpha}) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N y_i y_j \alpha_i \alpha_j \underline{\mathbf{x}}_i^T \underline{\mathbf{x}}_j$$

subject to

$$\underline{\alpha} \geq \underline{\mathbf{0}}, \sum_{i=1}^N \alpha_i y_i = 0$$
$$\underline{\mathbf{w}} = \sum_{i=1}^N \alpha_i y_i \underline{\mathbf{x}}_i$$

- The equation for $\underline{\mathbf{w}}$ is not a constraint, but it computes $\underline{\mathbf{w}}$ (the plane) given $\underline{\alpha}$, while b is given by $\min |\underline{\mathbf{w}}^T \underline{\mathbf{x}}_i + b| = 1$

Dual Form of SVM as QP Problem

- The dual form of SVM problem is a convex quadratic programming problem, in the form:

$$\begin{array}{ll}\text{minimize with respect to } \underline{\alpha} & \underline{\mathbf{1}}^T \underline{\alpha} - \frac{1}{2} \underline{\alpha}^T \underline{\mathbf{Q}} \underline{\alpha} \\ \text{subject to} & \underline{\alpha} \geq 0, \underline{\mathbf{y}}^T \underline{\alpha} = 0\end{array}$$

where:

- the matrix is $\underline{\mathbf{Q}} = \{y_i y_j \underline{\mathbf{x}}_i^T \underline{\mathbf{x}}_j\}_{ij}$
- $\underline{\alpha}$ is the column vector $(\alpha_1, \dots, \alpha_N)$

Solving Dual Formulation of SVM Problem (1/2)

- **Solving convex problem** for α
 - Feeding this problem to a QP solver, you get the optimal vector $\underline{\alpha}$
- **Compute hyperplane** $\underline{w}$
 - From $\underline{\alpha}$ recover the plane $\underline{w}$ from the equation: $\underline{w} = \sum_{i=1}^N \alpha_i y_i \underline{x}_i$
 - Looking at the optimal α_i , you can observe that many of them are 0
 - This is because when you applied the Lagrange multipliers to the inequalities: $y_i(\underline{w}^T \underline{x}_i + b) \geq 1$, you got the KKT condition:

$$\alpha_i(y_i(\underline{w}^T \underline{x}_i + b) - 1) = 0$$

- From these equations, either
 - * $\alpha_i = 0$ and $\underline{x}_i$ is an *interior point* since it has non-null distance from the plane (i.e., slack) from the plane; or
 - * $\alpha_i \neq 0$ and the slack is 0, which implies that the $\underline{x}_i$ point touches the margin, i.e., it is a *support vector*

Solving Dual Formulation of SVM Problem (2/2)

- Thus the hyperplane is only function of the support vectors:

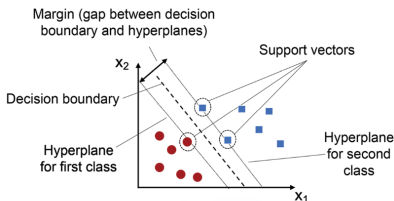
$$\underline{\mathbf{w}} = \sum_{i=1}^N \alpha_i y_i \underline{\mathbf{x}}_i = \sum_{\underline{\mathbf{x}}_i \in \text{SV}} \alpha_i y_i \underline{\mathbf{x}}_i$$

since only for the support vectors
 $\alpha \neq 0$

- The $\alpha_i \neq 0$ are the real degree of freedom

- Compute b**
 - Once $\underline{\mathbf{w}}$ is known, you can use any support vector to compute b :

$$y_i(\underline{\mathbf{w}}^T \underline{\mathbf{x}}_i + b) = 1$$



Support Vectors and Degrees of Freedom for SVM

- The number of support vectors is related to the degrees of freedom of the model
- Because of the VC dimension, you have an in-sample quantity to bound the out-of-sample error:

$$E_{out} \leq E_{in} + c \frac{\text{num of SVs}}{N - 1}$$

- You are “guaranteed” to not overfit

Non-Linear Transform for SVM

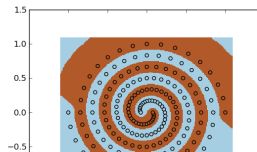
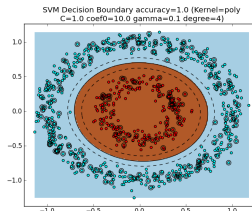
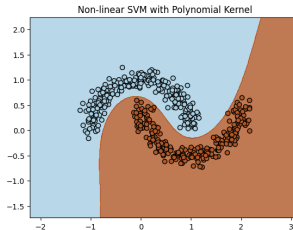
- $\Phi : \mathcal{X} \rightarrow \mathcal{Z}$ transforms $\underline{x}_i$ into $\underline{z}_i = \Phi(\underline{x}_i) \in \mathbb{R}^{\tilde{d}}$ with $\tilde{d} > d$
- Transform vectors through Φ and apply SVM machinery
- Dual SVM formulation in $\mathcal{Z}$ space:

$$\mathcal{L}(\underline{\alpha}) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j \underline{z}_i^T \underline{z}_j$$

- Note:
 - Optimization problem has same number of unknowns as original space (number of points N)
 - Support vectors live in $\mathcal{Z}$: they have $\alpha = 0$. In $\mathcal{X}$, they are pre-images of support vectors
 - Decision boundary and margin can be represented in original space (not linear)

Non-Linear Transforms for SVM vs Others

- In SVM the non-linear transform does not change the number of unknowns and degrees of freedom of the model
- This is different from transforming the variables in a linear problem, since in that case the number of unknowns changes



SVM in Higher Dimensional Space

■ Pros

- You don't pay the price in terms of complexity of optimization problem
 - * Number of unknowns is still N (different than a linear problem)
- You don't pay the price in terms of increased generalization bounds
 - * Number of support vectors is $\leq N$
 - * This is because each hypothesis h can be complex but the cardinality of the hypothesis set $\mathcal{H}$ is the same

■ Cons

- You pay a price to compute $\Phi(\underline{\mathbf{x}}_i)^T \Phi(\underline{\mathbf{x}}_j)$, since Φ could be very complex
 - * The kernel trick will remove this extra complexity by doing $\Phi(\underline{\mathbf{x}}_i)^T \Phi(\underline{\mathbf{x}}_j) = K_{\Phi}(\underline{\mathbf{x}}_i, \underline{\mathbf{x}}_j)$

Non-Linear Transform in SVM vs Kernel Trick

- The trivial approach is:
 - Transform vectors with $\Phi(\cdot)$
 - Apply all SVM machinery to the transformed vectors
 - **Cons:** Φ might be very complex, e.g., potentially exponential number of terms
- You can express the SVM problem formulation and the prediction in terms of a kernel

$$K_{\Phi}(\underline{\mathbf{x}}, \underline{\mathbf{x}}') = \Phi(\underline{\mathbf{x}})^T \Phi(\underline{\mathbf{x}}') = \underline{\mathbf{z}}^T \underline{\mathbf{z}}'$$

- You only need the kernel $K_{\Phi}(\underline{\mathbf{x}}, \underline{\mathbf{x}}')$ of the transformation $\Phi(\cdot)$ and not $\Phi(\cdot)$ itself

SVM in Terms of Kernel: Optimization Step

- When you build the QP formulation for the Lagrangian to compute the α we can use $K_{\Phi}(\underline{\mathbf{x}}_i, \underline{\mathbf{x}}_j)$ instead of $\underline{\mathbf{z}}_i^T \underline{\mathbf{z}}_j$

$$\mathcal{L}(\underline{\alpha}) = \sum_{n=1}^N \alpha_n - \frac{1}{2} \sum_{n=1}^N \sum_{m=1}^N y_n y_m \alpha_n \alpha_m K_{\Phi}(\underline{\mathbf{x}}_n, \underline{\mathbf{x}}_m)$$

- $\underline{\mathbf{z}}_n$ does not appear in the constraints

$$\underline{\alpha} \geq \underline{\mathbf{0}}, \underline{\alpha}^T \underline{\mathbf{y}} = 0$$

SVM in Terms of Kernel: Prediction Step

- You need only inner products to compute a prediction for a given $\underline{\mathbf{z}}$
- In fact to make predictions, you replace the expression of $\tilde{\underline{\mathbf{w}}} = \sum_{i:\alpha_i>0} \alpha_i y_i \underline{\mathbf{z}}_i$ in $h(\underline{\mathbf{x}}) = \text{sign}(\underline{\mathbf{w}}^T \Phi(\underline{\mathbf{x}}) + b)$, yielding:

$$h(\underline{\mathbf{x}}) = \text{sign}\left(\sum_{i:\alpha_i>0} \alpha_i y_i K_{\Phi}(\underline{\mathbf{x}}_i, \underline{\mathbf{x}}) + b\right)$$

where b is given by $y_i(\underline{\mathbf{w}}^T \underline{\mathbf{z}}_i + b) = 1$ for any support vector $\underline{\mathbf{x}}_m$ and thus

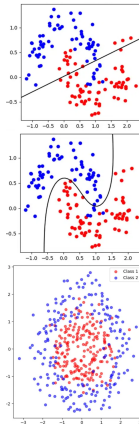
$$b = \frac{1}{y_m} - \sum_{i:\alpha_i>0} \alpha_i y_i K_{\Phi}(\underline{\mathbf{x}}_i, \underline{\mathbf{x}}_m)$$

Implications of Kernel Trick in SVM

- The “kernel trick” is a computational shortcut:
 - Use the kernel of the transformation instead of the transformation itself
- To use SVMs, compute inner products between transformed vectors $\underline{z}$
- The kernel trick implies:
 - No need to compute $\Phi()$
 - * Use the kernel K_Φ , not the transformation Φ
 - No need to know Φ
 - * With function K_Φ as an inner product, use SVM machinery without knowing $\mathcal{Z}$ space or transformation Φ
 - Φ can be impossible to compute
 - * K_Φ can correspond to a transformation Φ to an infinite dimensional space (e.g., Gaussian kernel)

Non-Linearly Separable SVM Problem

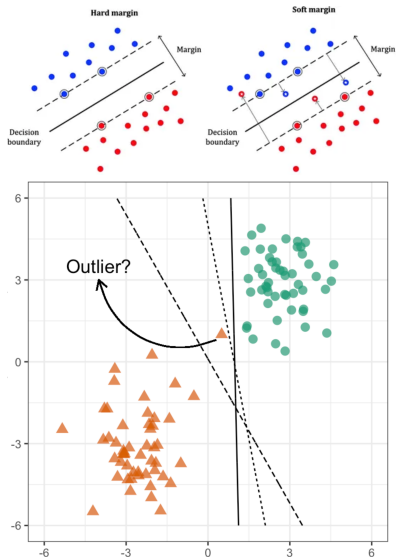
- In general there are different types of **non-linearly separable data sets**
- **Slightly non-separable**
 - Few points crossing the boundary
 - $\implies$ use soft margin SVMs
- **Seriously non-separable**
 - E.g., the class inside the circle
 - $\implies$ use non-linear kernels



- In practice, both issues are present
 - Combine soft margin SVM and non-linear kernel transforms

Soft-Margin SVM: Advantages

- Even with linearly separable data, improve E_{out} using soft margin SVM at the cost of worse E_{in}
 - Trade-off between in-sample and out-of-sample performance
- E.g., outliers force a smaller margin
 - Ignoring outliers could increase margin
- Large C parameter in SVM requires minimizing error, trading off large margin for correct classification



Primal Formulation for Soft Margin SVM

- You want to introduce an error measure based on the margin violation for each point, so instead of the constraint:

$$y_i(\underline{\mathbf{w}}^T \underline{\mathbf{x}}_i + b) \geq 1 \text{ (hard margin)}$$

- You can use:

$$y_i(\underline{\mathbf{w}}^T \underline{\mathbf{x}}_i + b) \geq 1 - \xi_i, \text{ where } \xi_i \geq 0 \text{ (soft margin)}$$

- The cumulative margin violation is $C \sum_{i=1}^N \xi_i$
- The soft margin SVM optimization (primal form) is:

$$\begin{aligned} & \text{find } \underline{\mathbf{w}}, b, \underline{\xi} \\ & \text{minimize} \quad \frac{1}{2} \underline{\mathbf{w}}^T \underline{\mathbf{w}} + C \sum_{i=1}^N \xi_i \\ & \text{subject to} \quad y_i(\underline{\mathbf{w}}^T \underline{\mathbf{x}}_i + b) \geq 1 - \xi_i \quad \forall i \\ & \quad \quad \quad \xi_i \geq 0 \end{aligned}$$

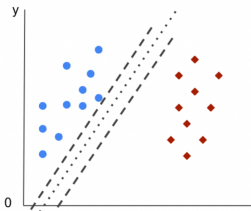
Classes of Support Vectors for Soft Margin SVM

- There are 3 classes of points:
 - **Margin support vectors**: they are exactly on the margin and define it
 - * In primal form: $y_i(\underline{\mathbf{w}}^T \underline{\mathbf{x}}_i + b) = 1 \iff \xi_i = 0$
 - * In dual form: $0 < \alpha_i < C$
 - **Non-margin support vectors**: they are inside the margin and classified correctly or not
 - * In primal form: $y_i(\underline{\mathbf{w}}^T \underline{\mathbf{x}}_i + b) < 1 \iff \xi_i > 0$
 - * In dual form: $\alpha_i = C$
 - **Non-support vectors**, i.e., interior points:
 - * In primal form: $y_i(\underline{\mathbf{w}}^T \underline{\mathbf{x}}_i + b) > 1$
 - * In dual form: $\alpha_i = 0$

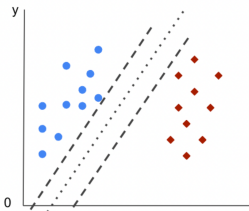
Intuition for C in SVM

- C represents how much penalty you incur for passing the margin
 - If C is large, then SVM will try to fit all the points to avoid being penalized
 - * Lower bias / higher variance
 - * $C \rightarrow \infty$ which yields hard-margin SVM
 - If C is small, then you allow margin violations
 - * Higher bias / lower variance
- From another point of view $C \propto \frac{1}{\lambda}$, so large C means small λ and thus small regularization
 - C is chosen through cross validation, like any regularization parameter

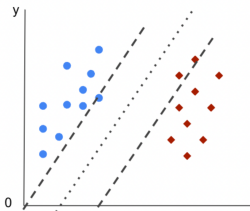
C = 100



C = 10



C = 1



Multi-Class Classification for SVM

- Often SVM packages have built-in multi-class classification
- Otherwise use the one-vs-all method:
 - Train K SVMs distinguishing each class from the rest using one-hot encoding
 - * Get SVM parameters $(\underline{\mathbf{w}}_1, b_1), \dots, (\underline{\mathbf{w}}_K, b_K)$
 - For a new example $\underline{\mathbf{x}}$ compute $\underline{\mathbf{w}}_i^T \underline{\mathbf{x}} + b_i$ for all the models
 - Pick the model that gives the largest positive value
 - * I.e., more confident about its class vs the rest of the classes